COMO REALIZAR LA DEPURACION

Paso 1: Reproduce el error

El primer paso para la depuración siempre es encontrar una serie de acciones que reproduzcan un error de manera uniforme.

1. Partiendo de este código trabaja con el depurador:

```
<!DOCTYPE html>
 <html>
 <body>
 <h1>Sumar dos números</h1>
 <form name="f1"
 <p>Por favor, introduce dos números:</p>
 <input name="num1" type="text"><br>
 <input name="num2" type="text">  <br>
 <input name="resul" type="text" disabled>
 <button type="button" onclick="myFunction(f1)">Sumar</button>
</form>
 <script language="javascript">
 function myFunction(f) {
   var x,y,suma,text;
   x =(f.num1.value);
   y =(f.num2.value);

   suma=x+y;
   text= suma;

   f.resul.value = text;
 }
 </script>
 </body>
 </html>
```

2. Ingresa `6` en el cuadro de texto **num1**
3. Ingresa `7` en el cuadro de texto **num2**.
4. Haz clic en **el botón Sumar**. La etiqueta debajo del botón dice `6 + 7 = 67`. El resultado debería ser `13`. Este es el error que vas a corregir.

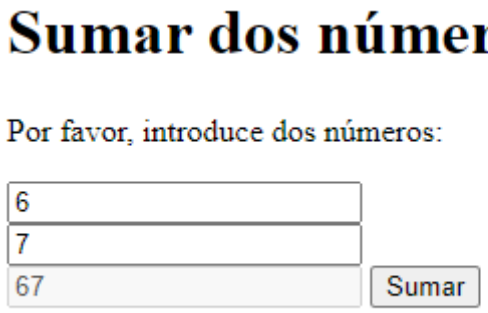


Figura 1. El resultado de 6 + 7 es 67. Debería ser 13.

Paso 2: Familiarízate con la IU del panel Sources

DevTools proporciona muchas herramientas diferentes para distintas tareas, como cambio de CSS, generación de perfiles de rendimiento de carga de páginas y supervisión de solicitudes de red. El panel **Sources** es donde depuras JavaScript.

1. Para abrir DevTools, presiona `Command`+`Option`+`I` (Mac) o `Ctrl`+`Mayús`+`I` (Windows, Linux). Este acceso directo abre el panel **Console**.

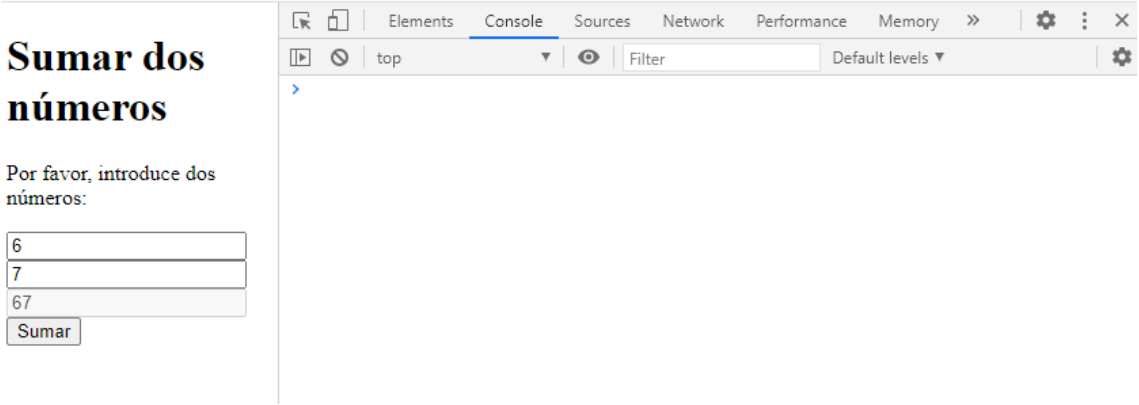


Figura 2. El panel **Console**

2. Haz clic en la pestaña **Sources**.

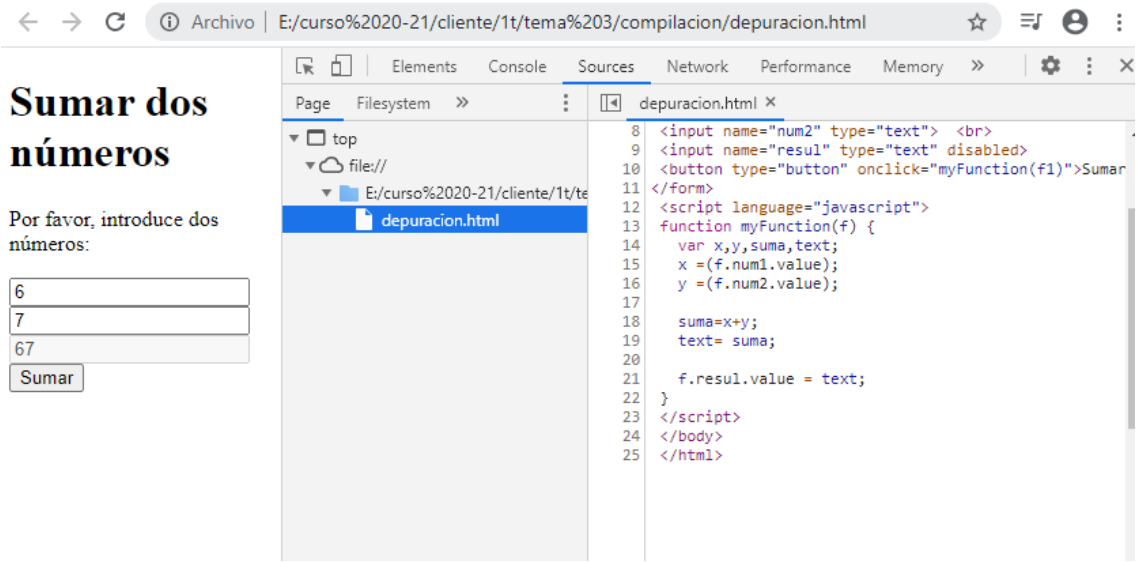


Figura 3. El panel **Sources**

La IU del panel **Sources** tiene 3 partes:

Figura 4. Las 3 partes de la IU del panel **Sources**

1. Panel **File Navigator**. Aquí se muestran todos los archivos que solicita la página.
2. Panel **Code Editor**. Después de seleccionar un archivo en el panel **File Navigator**, el contenido del archivo se muestra aquí.
3. Panel **JavaScript Debugging**. Diversas herramientas para inspeccionar el JavaScript de la página. Si la ventana de DevTools es ancha, este panel aparece a la derecha del panel **Code Editor**.

Paso 3: Pausa el código con un punto de interrupción

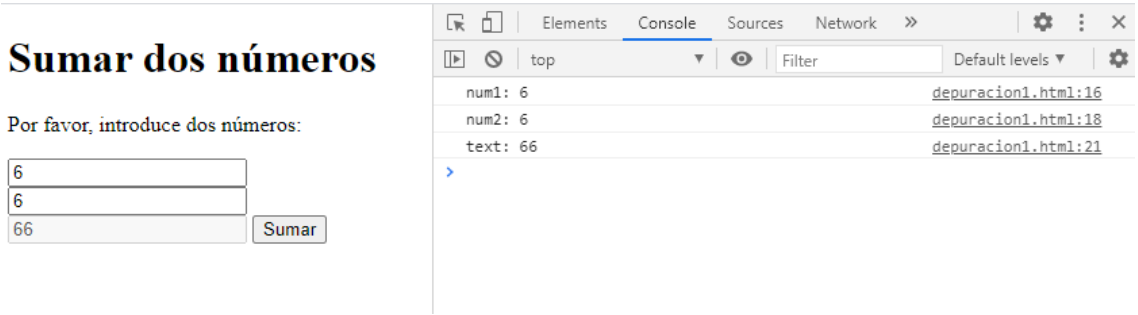
Un método común para depurar un problema como este es insertar muchas instrucciones de `console.log()` en el código para inspeccionar los valores a medida que se ejecuta la secuencia de comandos. Por ejemplo:

```
function myFunction(f) {
  var x,y,suma,text;
  x =(f.num1.value);
  console.log('num1:', x);
  y =(f.num2.value);
  console.log('num2:', y);
  suma=x+y;
  text= suma;
  console.log('text:', text);
  f.resul.value = text;
}
```

El método de `console.log()` es útil, pero los **puntos de interrupción** pueden ser más rápidos. Un punto de interrupción te permite pausar el código durante la ejecución y examinar todos los valores en ese momento específico. Los puntos de interrupción tienen algunas ventajas por sobre el método de `console.log()`:

- Con `console.log()`, debes abrir manualmente el código fuente, encontrar el código relevante, insertar las instrucciones de `console.log()` y volver a cargar la página para ver los mensajes en la consola. Con los puntos de interrupción, puedes realizar una pausa en el código relevante sin siquiera saber cómo está estructurado.
- En las instrucciones de `console.log()` debes especificar explícitamente cada valor que deseas inspeccionar. Con los puntos de interrupción, DevTools te muestra los valores de todas las variables en ese momento específico. A veces hay variables que afectan el código que ni siquiera sabes que existen.

En resumen, los puntos de interrupción pueden ser útiles para encontrar y corregir errores más rápido que con el método de `console.log()`.



Si piensas cómo funciona la app, puedes darte cuenta intuitivamente que se calcula la suma incorrecta (`6 + 6 = 66`) en el receptor de eventos `click` que está asociado con el botón **Sumar**. Por lo tanto, es probable que te convenga pausar el código aproximadamente en el momento de la ejecución del receptor `click`. **Event Listener Breakpoints** te permite hacer exactamente eso:

1. En el panel **JavaScript Debugging**, haz clic en **Event Listener Breakpoints** para expandir la sección. DevTools muestra una lista de categorías de eventos expandibles, como **Animation** y **Clipboard**.
2. Junto a la categoría de evento **Mouse**, haz clic en **Expand** ▶. DevTools muestra una lista de eventos de mouse, como **click** y **mousedown**. Cada evento tiene una casilla de verificación a su lado.
3. Marca la casilla de verificación **click**. DevTools ahora está configurado para hacer una pausa automáticamente cuando se ejecute *cualquier* receptor de eventos `click`.

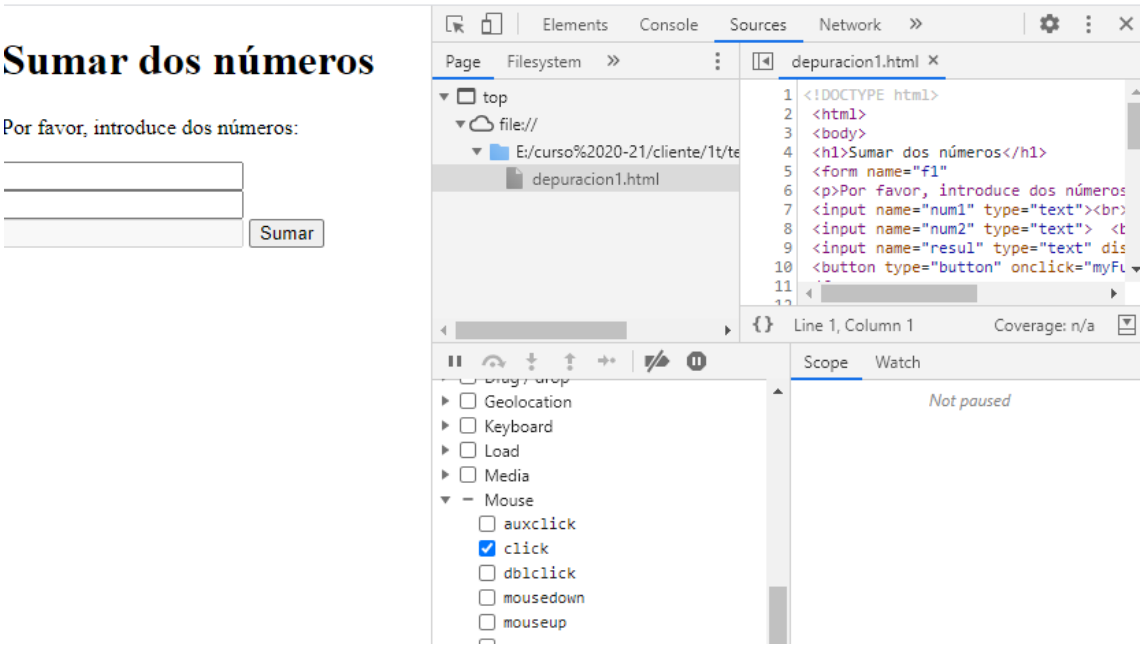


Figura 5. La casilla de verificación **click** está habilitada

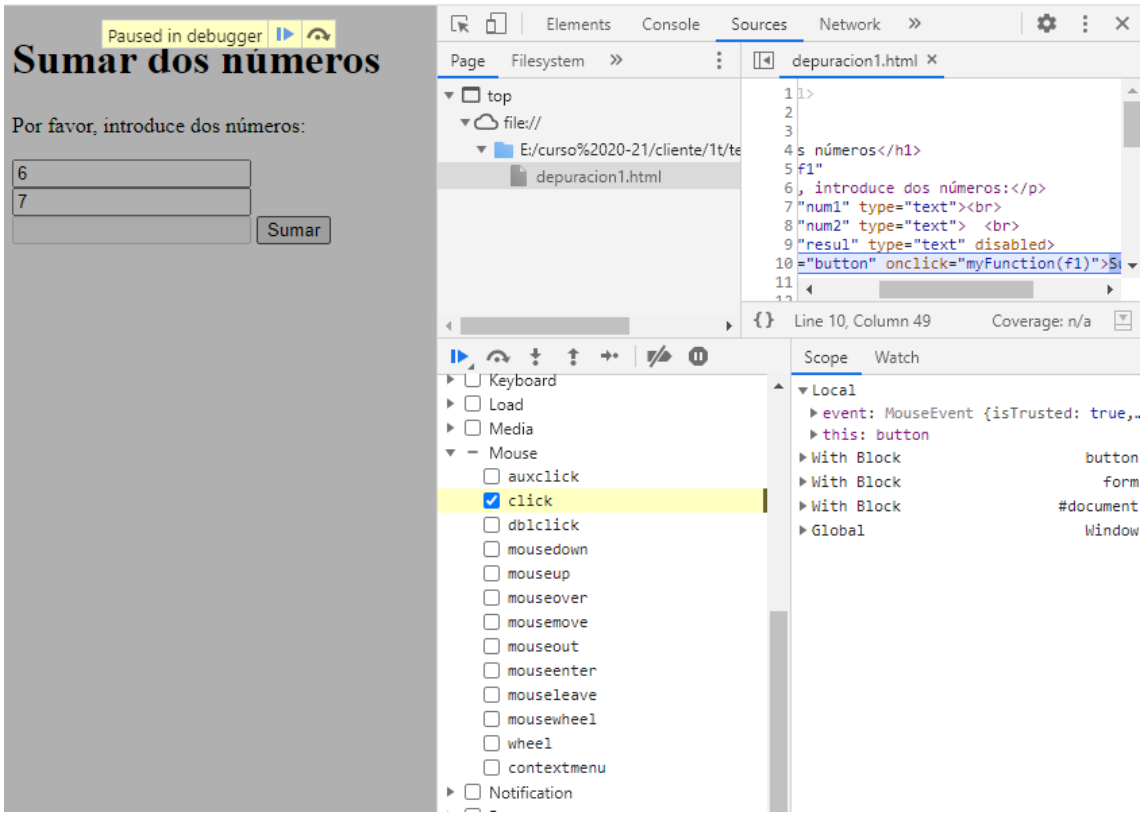
4. De regreso en la demostración, vuelve a hacer clic en **Sumar**. DevTools pausa la demostración y destaca una línea de código en el panel **Sources**. DevTools se debe pausar en esta línea de código:

```
function onClick() {
```

Si hiciste la pausa en una línea de código distinta, presiona **Resume Script Execution** hasta que el código quede pausado en la línea correcta.

Nota: Si pausaste en una línea distinta, tienes una extensión de navegador que registra un receptor de eventos `click` en cada página que visitas. Habías hecho una pausa en el receptor `click` de la extensión. Si usas el modo de navegación incógnito para navegación privada, que deshabilita todas las extensiones, puedes ver que la pausa siempre se hace en la línea de código correcta.

Los **puntos de interrupción de receptor de eventos** son uno de varios tipos de puntos de interrupción disponibles en DevTools. Es útil memorizar todos los tipos que existen porque cada uno te ayudará a depurar distintas situaciones con la mayor rapidez posible. Consulta Puntos de interrupción para pausas en el código para obtener información sobre cuándo y cómo usar cada tipo.



Paso 4: Recorre el código

Una causa común de errores es que la secuencia de comandos se ejecute en el orden incorrecto. Si recorres el código, podrás seguir la ejecución, línea por línea, y conocer exactamente qué parte se ejecuta en un orden distinto del esperado. Inténtalo ahora:

1. En el panel **Sources** de DevTools, haz clic en **Step into next function call** para recorrer la ejecución de la función `onClick()` una línea a la vez.
2. Haz clic en **Step over next function call** .

Paso 5: Establece un punto de interrupción de línea de código

El tipo de punto de interrupción más común es el de línea de código. Cuando encuentres una determinada línea de código en la que quieras pausar, usa un punto de interrupción de línea de código:

1. Observa la última línea de código de `la función`:

```
f.resul.value = text;
```

2. A la izquierda del código puedes ver el número de línea de esta línea de código específica, que es **17**. Haz clic en 17. DevTools coloca un ícono azul sobre **17**. Esto significa que la línea tiene un punto de interrupción de línea de código. DevTools ahora siempre hace una pausa antes de que se ejecute esta línea de código.
3. Haz clic en **Resume script execution** . La secuencia de comandos se sigue ejecutando hasta llegar a la línea 17. En las líneas 18 y 19, DevTools muestra los valores de `x`, `y` y `suma` a la derecha del punto y coma de cada línea.
4.

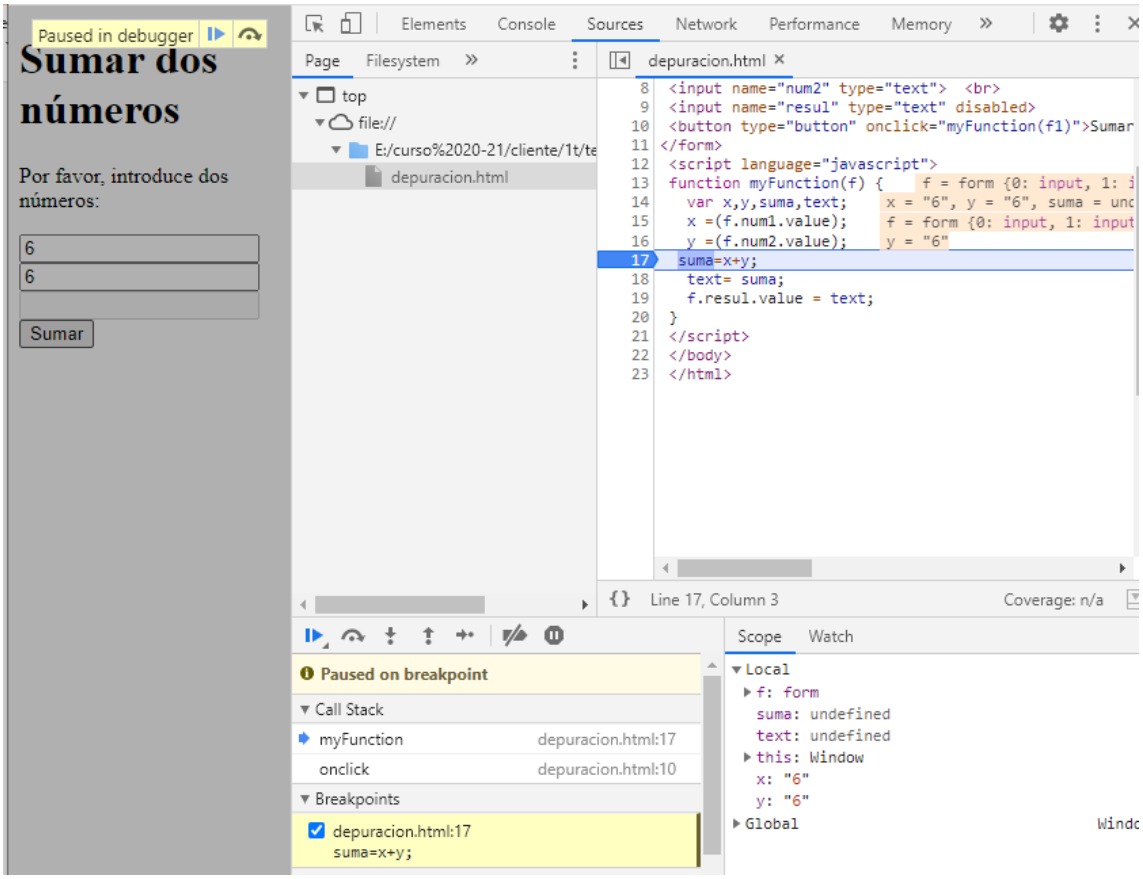


Figura 6. DevTools pausa en el punto de interrupción de línea de código en la línea 17

Paso 6: Verifica los valores de las variables

Los valores de `x`, `y` y `suma` se ven sospechosos. Están entre comillas, lo que significa que son strings. Esta es una buena hipótesis para explicar la causa del error. Ahora debes recopilar más información. DevTools proporciona muchas herramientas para examinar valores de variables.

Método 1: El panel Scope

Cuando el código está pausado en una línea, el panel **Scope** te muestra las variables locales y globales actualmente definidas, junto con el valor de cada variable. También muestra variables de cierre, cuando corresponde. Haz doble clic en un valor de variable para editarlo. Cuando el código no está pausado en una línea, el panel **Scope** está vacío.

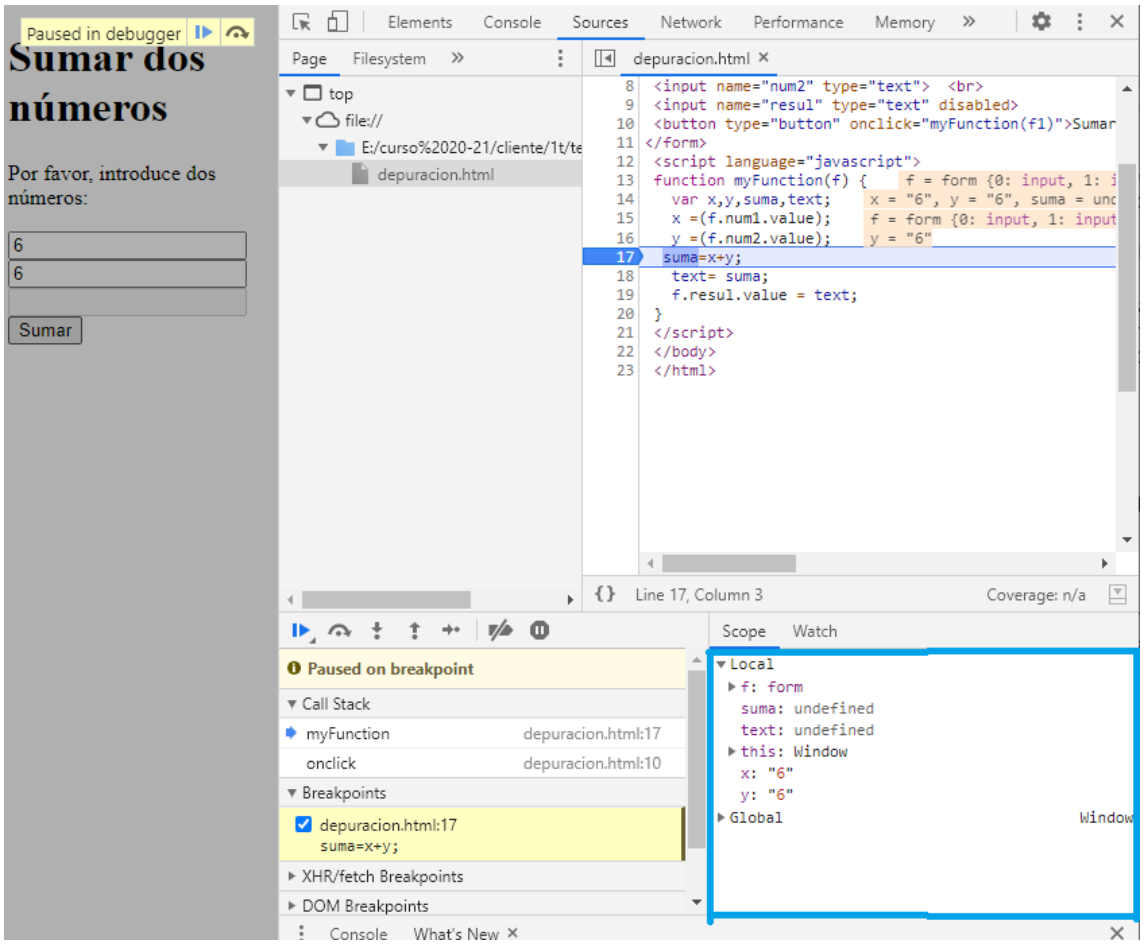


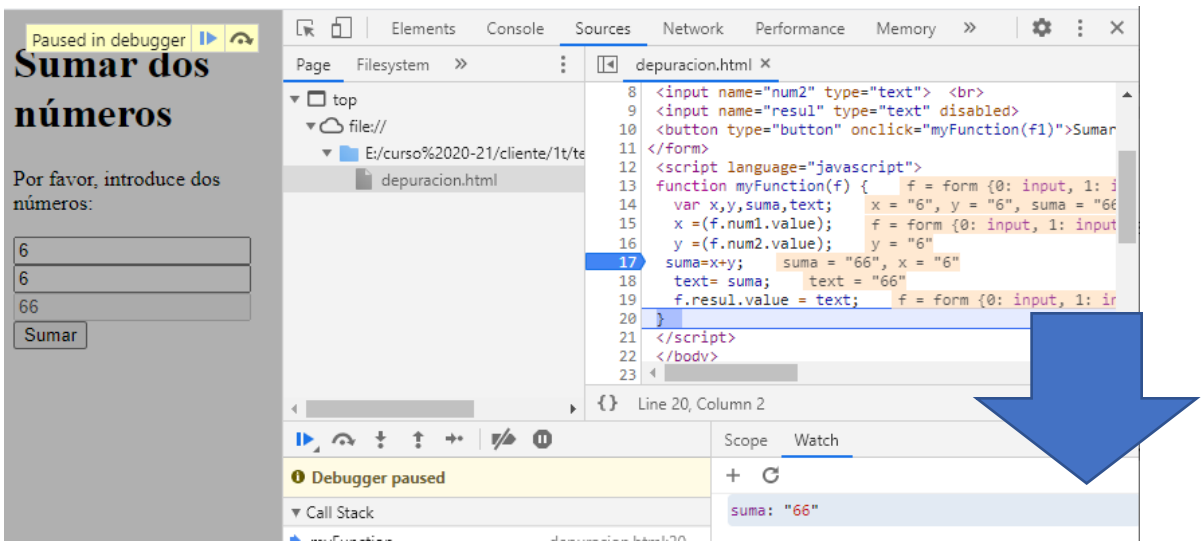
Figura 7. Panel **Scope**

Método 2: Expresiones supervisadas

La pestaña **Watch** te permite supervisar los valores de las variables a lo largo del tiempo. Como su nombre lo indica, las expresiones supervisadas no se limitan a las variables. En una expresión supervisada, puedes almacenar cualquier expresión válida de JavaScript. Inténtalo ahora:

1. Haz clic en la pestaña **Watch**.
2. Haz clic en **Add Expression** .
3. Escribe `typeof suma`.

4. Presiona `Intro`. DevTools muestra `typeof sum: "string"`. El valor a la derecha de los dos puntos es el resultado de la expresión supervisada.



Tengo que pulsar el + y añadir como se llama la variable

Figura 8. Panel de expresiones supervisadas (abajo a la derecha) después de crear la expresión supervisada `typeof suma`. Si la ventana de DevTools es grande, el panel de expresiones supervisadas se encontrará a la derecha, arriba del panel **Event Listener Breakpoints**.

Como sospechábamos, `suma` se evalúa como una string en lugar de un número. Has confirmado que esta es la causa del error.

Método 3: La consola

Además de ver los mensajes de `console.log()`, también puedes usar la consola para evaluar instrucciones arbitrarias de JavaScript. En términos de depuración, puedes usar la consola para probar correcciones potenciales de errores. Inténtalo ahora:

1. Si el panel lateral Console no está abierto, presiona `Escape` para abrirlo. Se abrirá en la parte inferior de la ventana de DevTools.
2. En la consola, escribe `parseInt(x) + parseInt(y)`. Esta instrucción funciona porque el código está pausado en una línea donde `x` y `y` están dentro del alcance.
3. Presiona `Intro`. DevTools evalúa la instrucción y muestra `6`, que es el resultado esperado de la demostración.

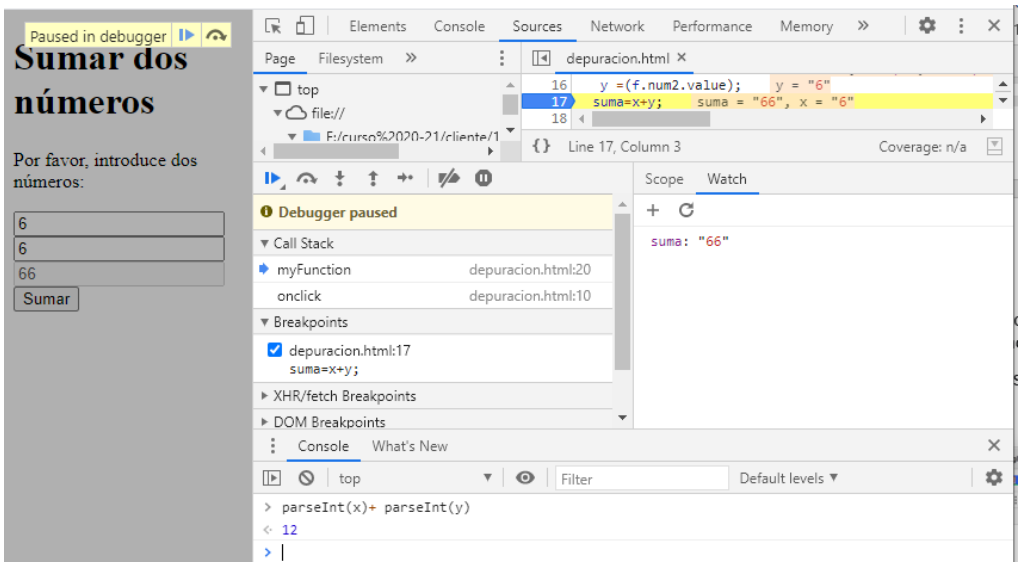
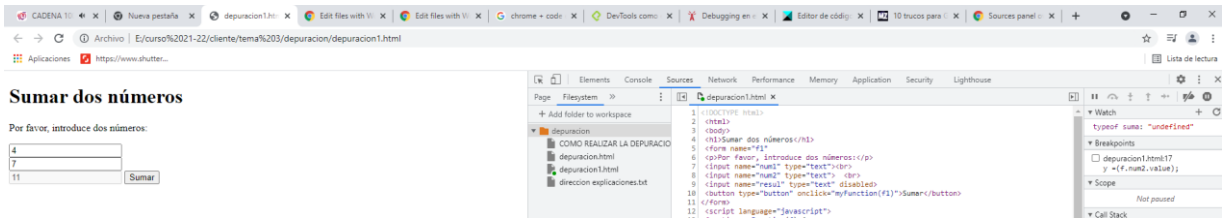


Figura 9. El panel lateral Console después d [illegible] x) + `parseInt(y)`.

Paso 7: Soluciona el problema

Encontraste una solución para el error. Lo único que falta es probar la solución. Para hacerlo, edita el código y vuelve a ejecutar la demostración. No hace falta salir de DevTools para insertar la solución. Puedes editar el código JavaScript directamente en la IU de DevTools. Inténtalo ahora:

1. Haz clic en **Resume script execution** .
2. En **Code Editor**, reemplaza la línea 19, `var sum = x + y`, con `var sum = parseInt(x) + parseInt(y)`. Para ello debo irme al panel 1, Filesystem y agregar la carpeta donde este el código. En el navegador me pedirá permiso para poder acceder desde el navegador a la página, se lo doy y ya puedo editar directamente desde el editor.



3. Presiona `Command+S` (Mac) o `Ctrl+S` (Windows, Linux) para guardar el cambio.
4. Haz clic en **Deactivate breakpoints** . Cambia a azul para indicar que está activo. Mientras esté configurado así, DevTools ignorará los puntos de interrupción que hayas establecido.
5. Prueba la demostración con distintos valores. Ahora los cálculos se realizan de manera correcta.

Warning: Este flujo de trabajo solo corrige el código que se ejecuta en tu navegador. No corregirá el código para todos los usuarios que visiten tu página. Para eso, debes corregir el código que está en los servidores.

Próximos pasos

¡Felicitaciones! Puedes aprovechar al máximo Chrome DevTools cuando depuras JavaScript. Las herramientas y los métodos que aprendiste en este instructivo pueden ahorrarte muchísimas horas.

En este instructivo, solo se mostraron dos formas de colocar puntos de interrupción. DevTools ofrece muchas otras formas, como las siguientes:

- Puntos de interrupción condicionales que solo se activan cuando la condición que proporciones sea verdadera
- Puntos de interrupción en excepciones detectadas y no detectadas
- Puntos de interrupción XHR que se activan cuando la URL solicitada coincide con una subcadena que proporciones